



Team name: Telementors

Shen, Multimodal RAG Platform

Repo Link – <https://github.com/Telementors/Shen-Multimodal-RAG-Platform>

SLIIT Codefest 2026: AI Competition

(Intelligent Document Assistant, powered by IFS)

Sub-track 1A: Rich Answers, Not Just Text

Problem Statement & Chosen subtrack

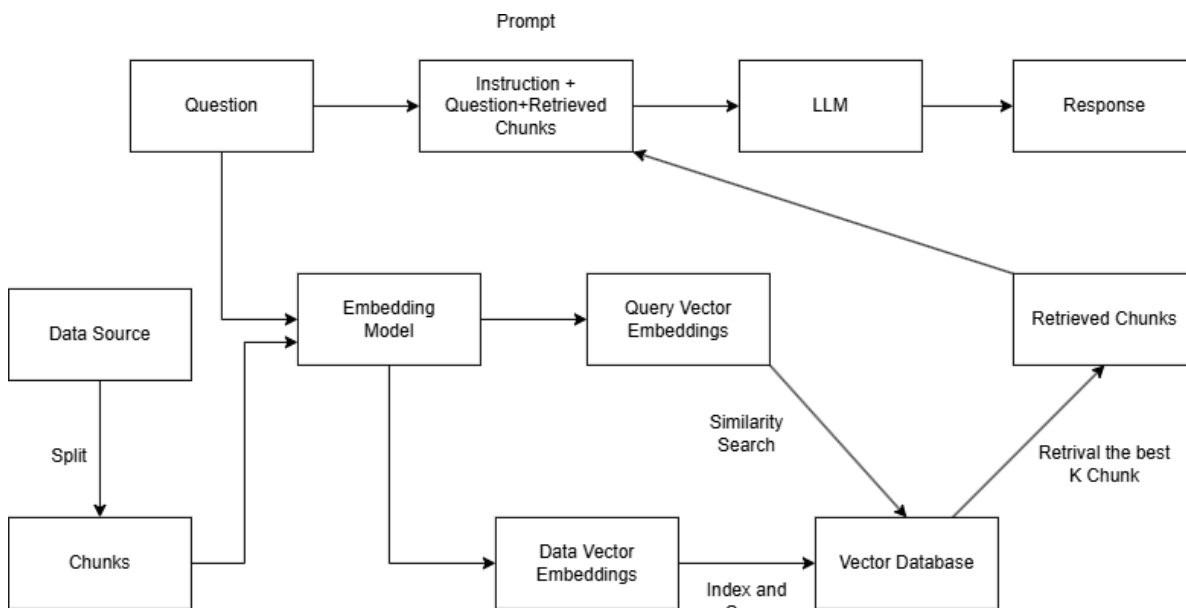
Technical documentation: manuals, datasheets, research papers, and lore archives routinely mix narrative prose with tables and diagrams, and much of the information a reader actually needs lives in those tables and figures rather than in the surrounding text. Conventional text-only RAG pipelines either drop non-text content entirely or flatten everything into a single index, where tables and figures get drowned out by the much larger volume of narrative text. The result is shallow, text-only answers that miss information locked in visual or tabular content, with no way to show the reader the source diagram or table that actually answers their question.

Our team selected **Sub-track 1A, Rich Multimodal Answers**: building a document assistant that returns answers embedding the actual supporting diagrams, tables, and images in line with citations to the source document and page rather than only describing them in prose.

We validated the system against the **Ashen Era Archive**, a purpose-built fictional multimodal corpus of 341 files (chronicles, lore codexes, in-world ephemera, wiki articles, and standalone figure plates) designed to mirror real technical-documentation challenges, including deliberately unreliable in-world sources used to test how the system handles conflicting information.

Solution Overview & Architecture

Shen is a RAG platform that maintains three **separate vector indices**: text, table, and image instead of one merged index, and uses an **LLM-powered agentic router** to classify each incoming query (text / table / image / hybrid) and search only the relevant modality. Retrieved chunks are composed into a single grounded answer with inline `[[IMAGE:N]]` and `[[TABLE:N]]` markers, rendered by the frontend as structured, reading-order answer blocks so the reader sees the actual figure or table, not just a description of it.

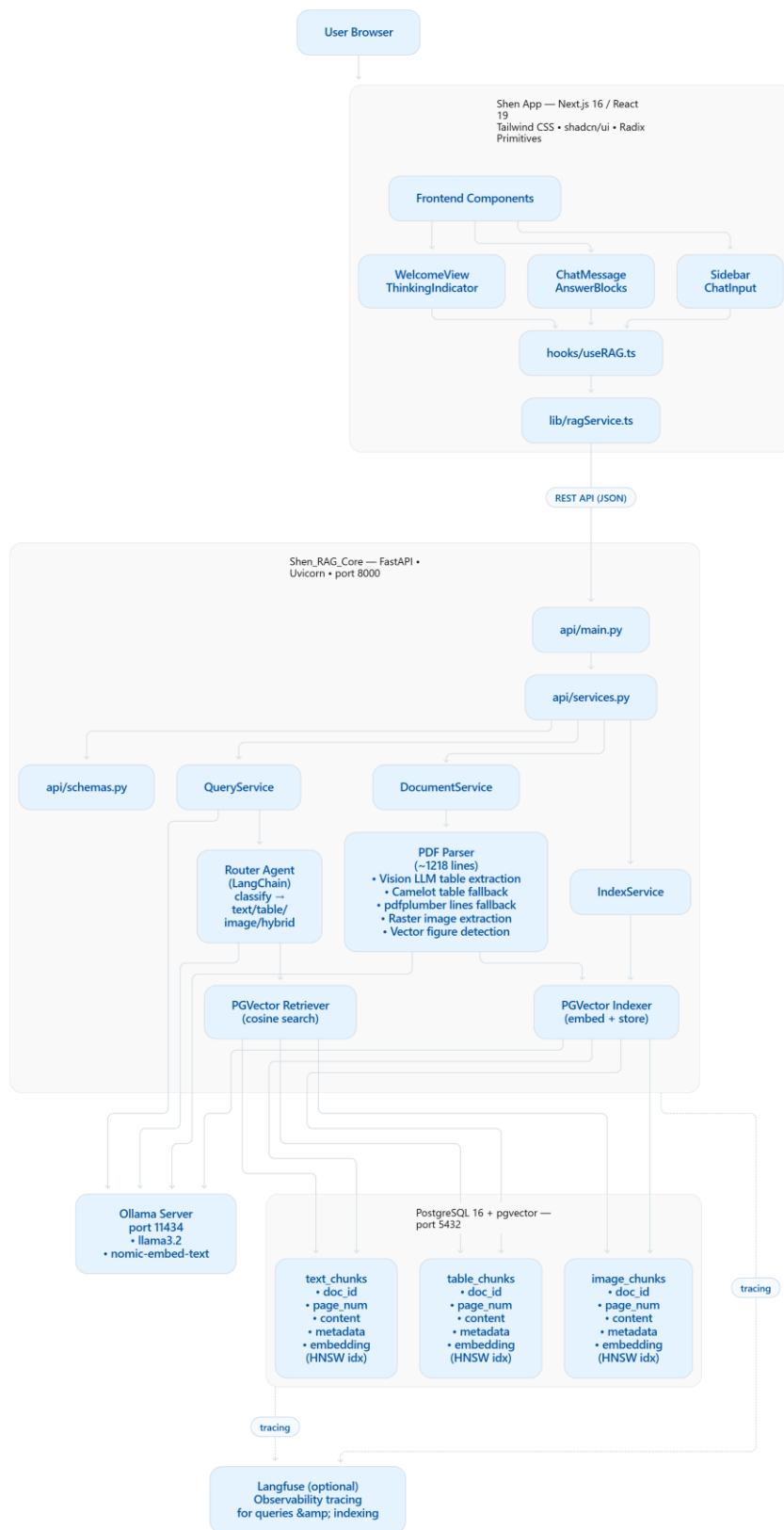


Layer	Technology
Frontend	Next.js 16 / React 19, Tailwind CSS
Backend	FastAPI + Uvicorn, Python 3.11
PDF parsing	unstructured, pdfplumber, PyMuPDF, Camelot vision-first 4-stage table cascade
LLM backend	Ollama
Observability	Langfuse
AI / ML	LangChain, langchain-openai, langchain-core
Database	PostgreSQL 16, pgvector, psycopg2
DevOps	Docker, Docker Compose, Node.js 20
Development Tools	GitHub Copilot, Claude (Anthropic), Google Gemini / Antigravity, Git

How It Works

Indexing pipeline	Query pipeline
1. PDF pages are parsed page-by-page into typed blocks (heading / text / table / image) via unstructured. 2. Tables run through the 4-stage vision-first cascade; raster images are pulled via PyMuPDF and vector figures (charts, diagrams) are detected by clustering drawing paths and anchoring to captions. 3. Table rows are converted to natural-language descriptions (LLM-summarised or deterministic). 4. Each block is embedded and stored in its matching pgvector table, HNSW-indexed for cosine search.	1. The LLM router classifies the question as text / table / image / hybrid and returns a short reasoning string alongside the answer. 2. The retriever runs cosine similarity search on the relevant table(s); hybrid queries search all three and merge results by similarity score. 3. The LLM composes a cited answer, embedding [[IMAGE:N]] / [[TABLE:N]] markers inline. 4. The response is parsed into structured answer blocks in reading order; retrieved media the LLM didn't reference is auto-appended.

Detailed Component Architecture



Key Technical Decisions

- Separate indices per modality: a single merged index lets tables and figures get drowned out by the much larger volume of narrative text; three independent pgvector indices give each modality fair representation during retrieval.
- LLM router over keyword heuristics: keyword rules are brittle on ambiguous questions; an LLM router understands intent and context, with a JSON-fallback parser so routing stays robust even on smaller local models.

- Dual LLM backend (Ollama + OpenAI): supporting both a fully local/zero-cost backend and a cloud backend behind one OpenAI-compatible interface means switching is an environment-variable change, not a code change.
- PostgreSQL + pgvector over a dedicated vector database: keeps deployment to a single database, reduces infrastructure complexity, and HNSW indexing gives competitive performance at the moderate scale this system targets.
- Vision-first, 4-stage table extraction cascade (Vision LLM → Camelot → pdfplumber lines → pdfplumber default): maximises extraction quality on edge cases such as LaTeX math and merged cells, with each stage acting as a fallback for the one before it.
- Structured rich answer blocks: returning typed AnswerBlock objects lets the frontend render images and tables inline, in reading order, and auto-append logic guarantees retrieved media is shown even if the LLM forgets to reference it.

What works, What doesn't and Limitations

What works

- Agentic routing across text, table, and image indices, with a reasoning string returned alongside every answer for transparency.
- The 4-stage vision-first table extraction cascade successfully handles complex tables, including LaTeX-style math and merged cells.
- Dual LLM backend is fully operational: the system runs end-to-end either fully local and free (Ollama) or against OpenAI for higher-quality answers.
- A golden-dataset evaluation harness (token-overlap F1, routing accuracy, optional LLM-as-judge) runs against the live query endpoint.
- The full 341-file Ashen Era Archive corpus is auto-indexed in the background on startup, across all three modalities.

Approaches we tried and failed

- We initially planned to run all LLM calls through OpenRouter's free tier. The free API key only came with a limited credit allowance, which was not enough to sustain development, indexing, and testing across the full corpus.
- We resolved this by standing up a local Ollama server (llama3.2 for chat/routing, nomic-embed-text for embeddings) as the default backend, removing the dependency on a rate-limited external API at some cost to answer quality relative to GPT-4o-mini.

Known limitations

- Input format: PDF parsing is the primary pipeline; DOCX files in the corpus are read but not parsed through the same vision-first cascade.
- Scale: tuned for moderate collections (hundreds of documents); HNSW performance may degrade at millions of vectors, and the single-node PostgreSQL deployment limits horizontal scaling.
- Language: optimised for English; multilingual documents and non-English table layouts see reduced retrieval quality.
- Hybrid-query latency: hybrid queries search all three tables sequentially with no result caching, adding latency versus a single-modality query.
- Image understanding: image chunks rely on LLM-generated vision descriptions and surrounding text context rather than a native vision-embedding model, so complex diagrams with minimal annotation may be poorly indexed.
- Local model quality: the zero-cost Ollama/llama3.2 backend produces lower-quality answers than GPT-4o-mini; this is an explicit cost/quality trade-off, not a bug.
- Evaluation coverage: the golden dataset covers 50 source documents, so it may not capture every real-world edge case, and LLM-as-judge scoring carries inherent subjectivity.